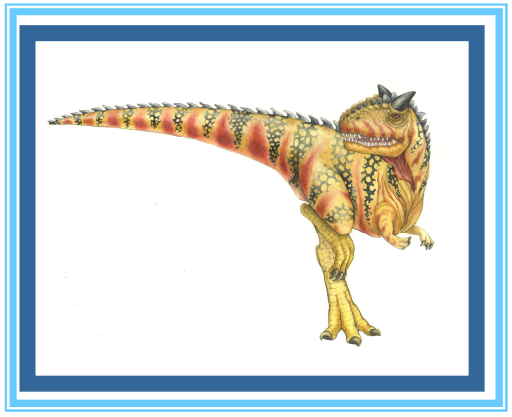
CPU Scheduling



Silberschatz, Galvin and Gagne ©2009



Contents

Scheduling Criteria

Multilevel Feedback Queue Scheduling

Thread Scheduling





CPU Scheduler

Selects from among the processes in memory that are ready to execute, and allocates the CPU to one of them

CPU scheduling decisions may take place when a process:

1. Switches from running to waiting state
2. Switches from running to ready state
3. Switches from waiting to ready
4. Terminates

Scheduling under 1 and 4 is **nonpreemptive**

All other scheduling is **preemptive**





Dispatcher

Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:

- switching context
- switching to user mode
- jumping to the proper location in the user program to restart that program

Dispatch latency – time it takes for the dispatcher to stop one process and start another running





Scheduling Criteria

CPU utilization – keep the CPU as busy as possible

Throughput – # of processes that complete their execution per time unit

Turnaround time – amount of time to execute a particular process

Waiting time – amount of time a process has been waiting in the ready queue

Response time – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)





Multilevel Queue

Ready queue is partitioned into separate queues:
foreground (interactive)
background (batch)

Each queue has its own scheduling algorithm

foreground – RR

background – FCFS

Scheduling must be done between the queues

Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.

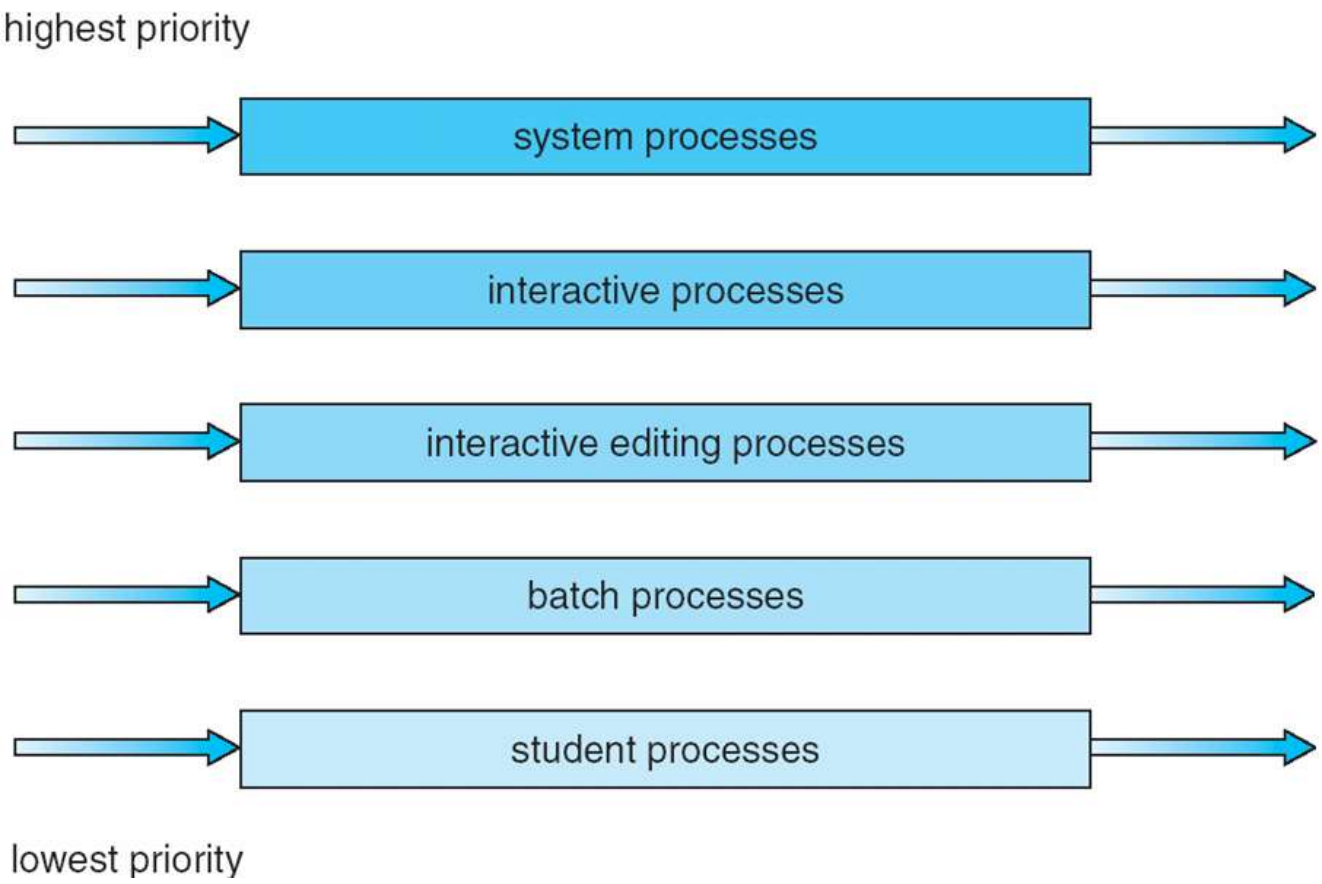
Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; for example:

- 80% to foreground in RR
- 20% to background in FCFS





Multilevel Queue Scheduling







Multilevel Feedback Queue

A process can move between the various queues; aging can be implemented this way

Multilevel-feedback-queue scheduler defined by the following parameters:

- number of queues
- scheduling algorithms for each queue
- method used to determine when to upgrade a process
- method used to determine when to demote a process
- method used to determine which queue a process will enter when that process needs service





Example of Multilevel Feedback Queue

Three queues:

- Q_0 – RR with time quantum 8 milliseconds
- Q_1 – RR time quantum 16 milliseconds
- Q_2 – FCFS

Scheduling

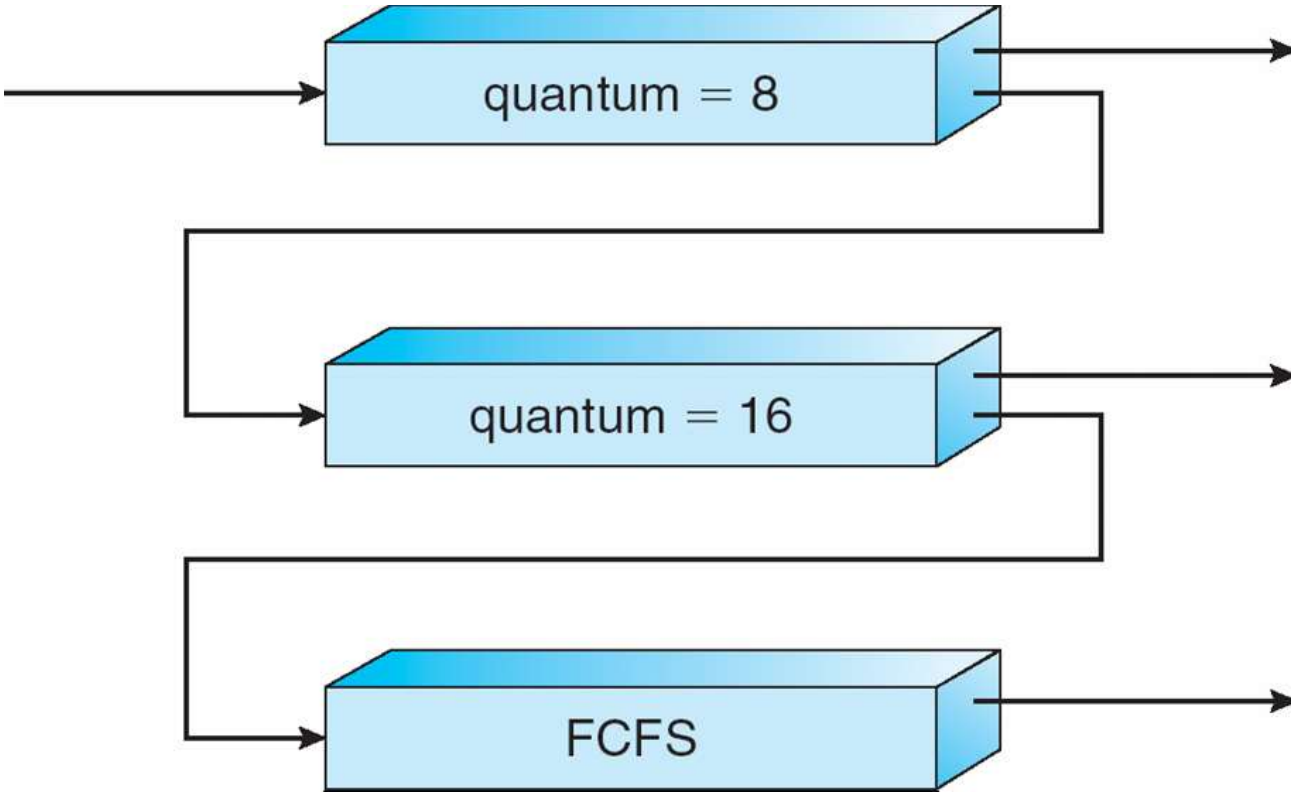
A new job enters queue Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1.

At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2.





Multilevel Feedback Queues







Multiple-Processor Scheduling

CPU scheduling more complex when multiple CPUs are available

Homogeneous processors within a multiprocessor

Asymmetric multiprocessing – only one processor accesses the system data structures, alleviating the need for data sharing

Symmetric multiprocessing (**SMP**) – each processor is self-scheduling, all processes in common ready queue, or each has its own private queue of ready processes

- Currently, most common

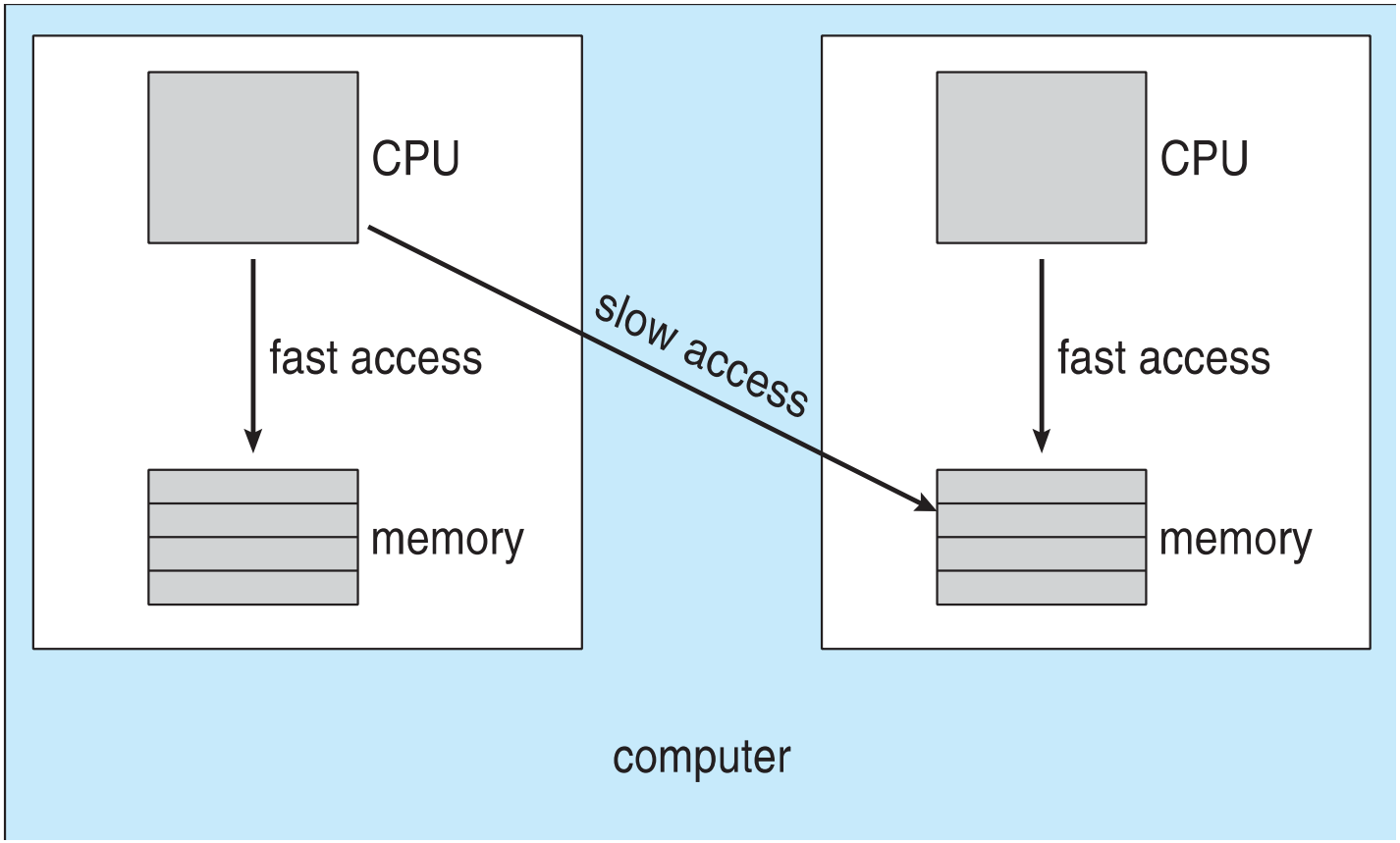
Processor affinity – process has affinity for processor on which it is currently running

- **soft affinity**
- **hard affinity**
- Variations including **processor sets**





NUMA and CPU Scheduling



Note that memory-placement algorithms can also consider affinity





Multiple-Processor Scheduling – Load Balancing

If SMP, need to keep all CPUs loaded for efficiency

Load balancing attempts to keep workload evenly distributed

Push migration – periodic task checks load on each processor, and if found pushes task from overloaded CPU to other CPUs

Pull migration – idle processors pulls waiting task from busy processor





Multicore Processors

Recent trend to place multiple processor cores on same physical chip

Faster and consumes less power

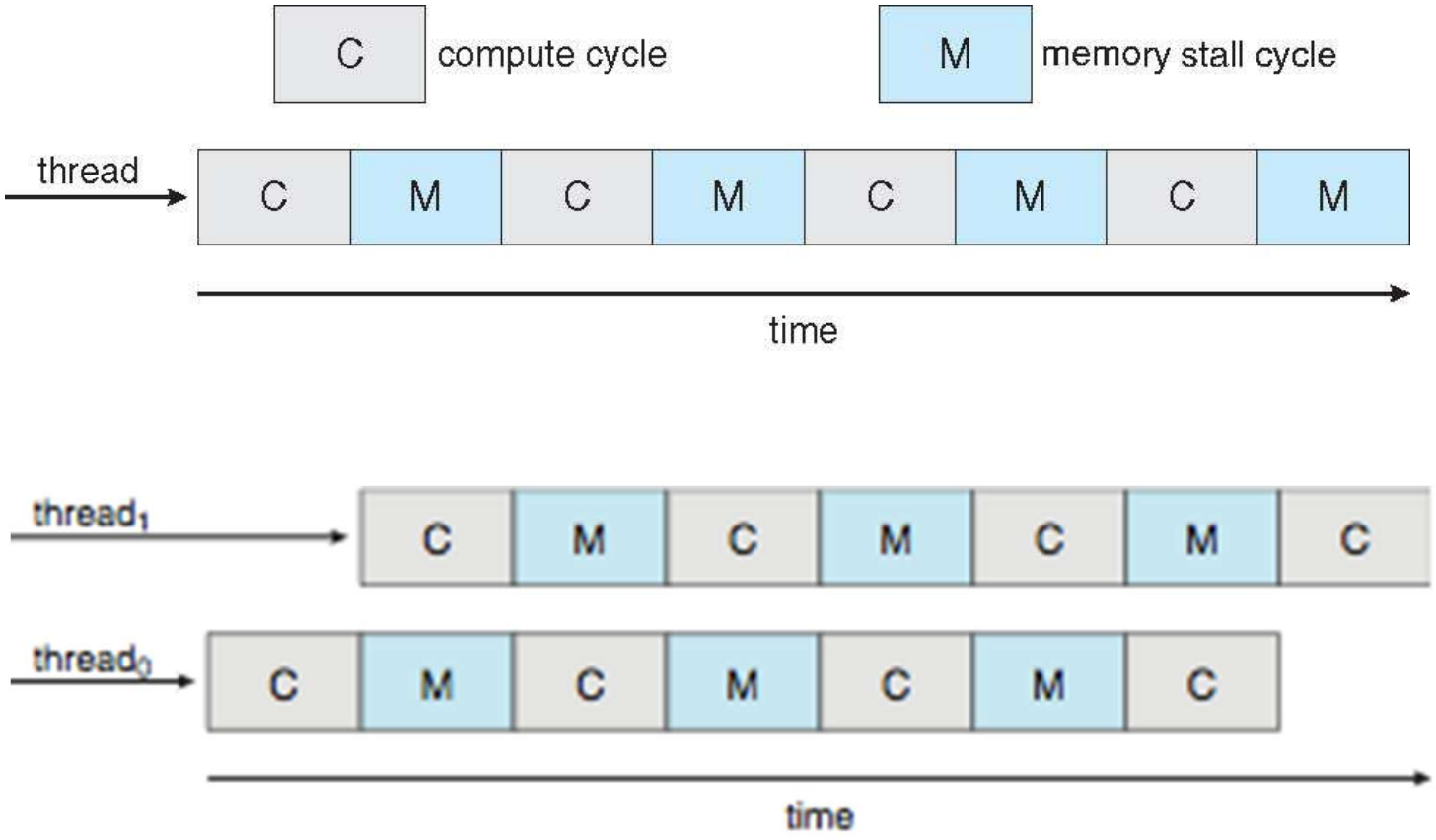
Multiple threads per core also growing

- Takes advantage of memory stall to make progress on another thread while memory retrieve happens





Multithreaded Multicore System







Virtualization and Scheduling

Virtualization software schedules multiple guests onto CPU(s)

Each guest doing its own scheduling

- Not knowing it doesn't own the CPUs
- Can result in poor response time
- Can effect time-of-day clocks in guests

Can undo good scheduling algorithm efforts of guests





Real-Time CPU Scheduling

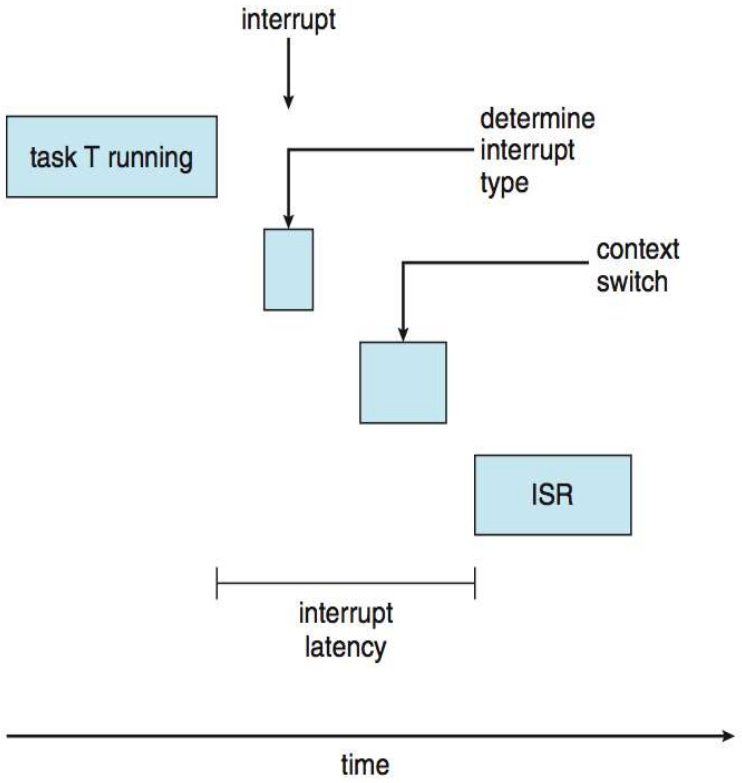
Can present obvious challenges

Soft real-time systems – no guarantee as to when critical real-time process will be scheduled

Hard real-time systems – task must be serviced by its deadline

Two types of latencies affect performance

1. Interrupt latency – time from arrival of interrupt to start of routine that services interrupt
2. Dispatch latency – time for schedule to take current process off CPU and switch to another



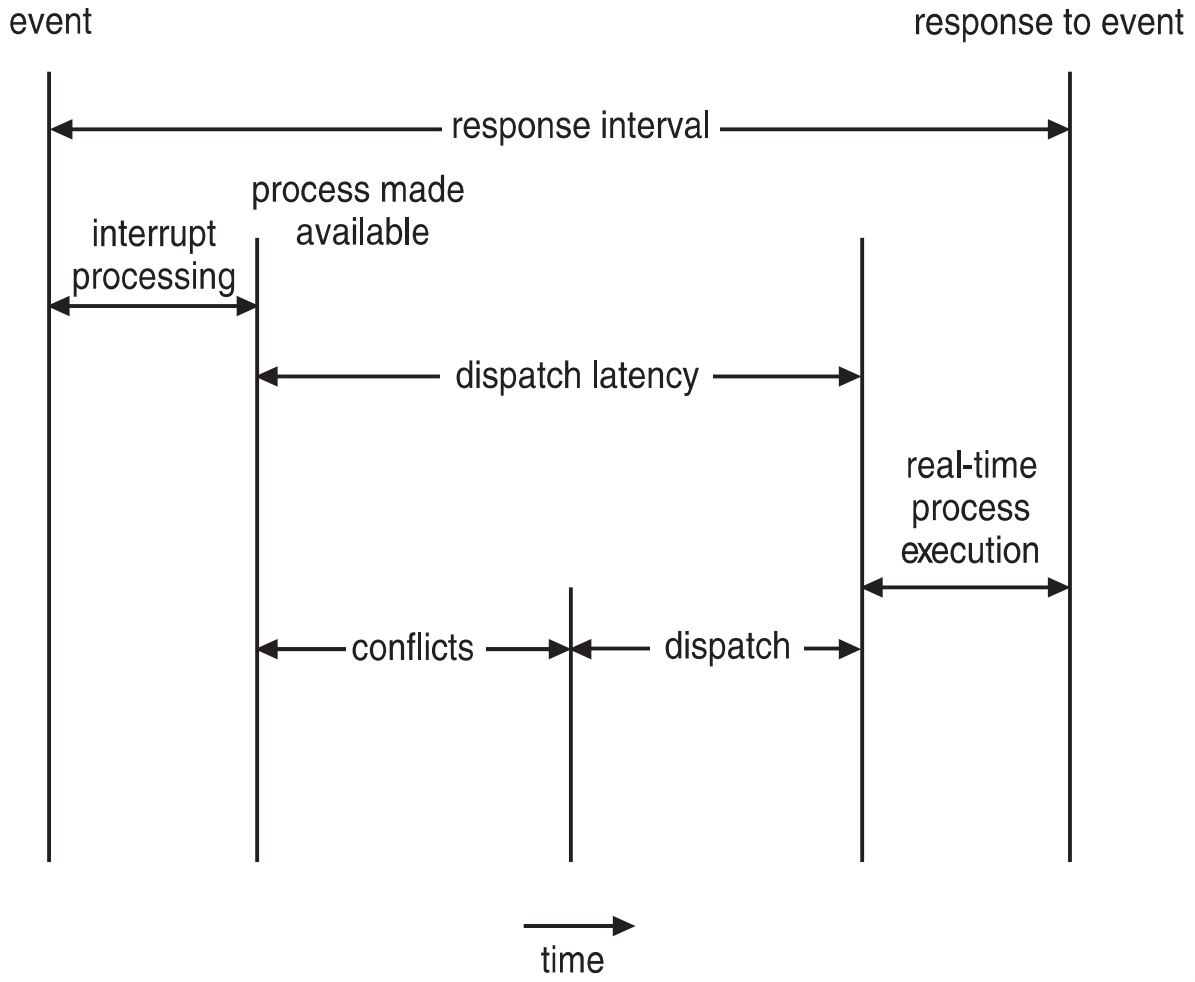




Real-Time CPU Scheduling (Cont.)

Conflict phase of dispatch latency:

1. Preemption of any process running in kernel mode
2. Release by low-priority process of resources needed by high-priority processes







Priority-based Scheduling

For real-time scheduling, scheduler must support preemptive, priority-based scheduling

But only guarantees soft real-time

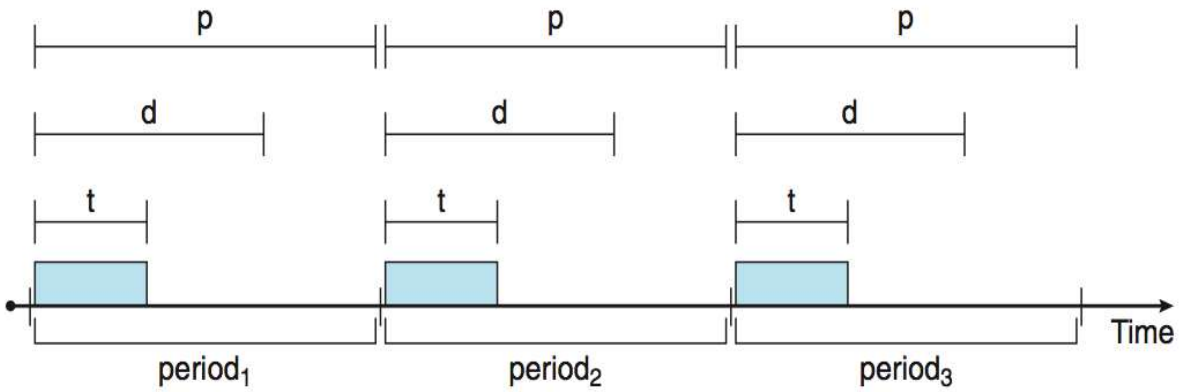
For hard real-time must also provide ability to meet deadlines

Processes have new characteristics: **periodic** ones require CPU at constant intervals

Has processing time *t*, deadline *d,* period *p*

0

R







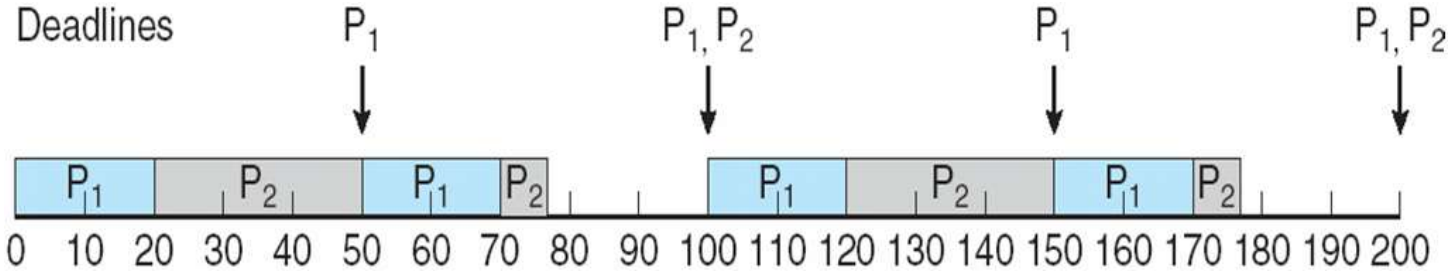
Rate Montonic Scheduling

A priority is assigned based on the inverse of its period

Shorter periods = higher priority;

Longer periods = lower priority

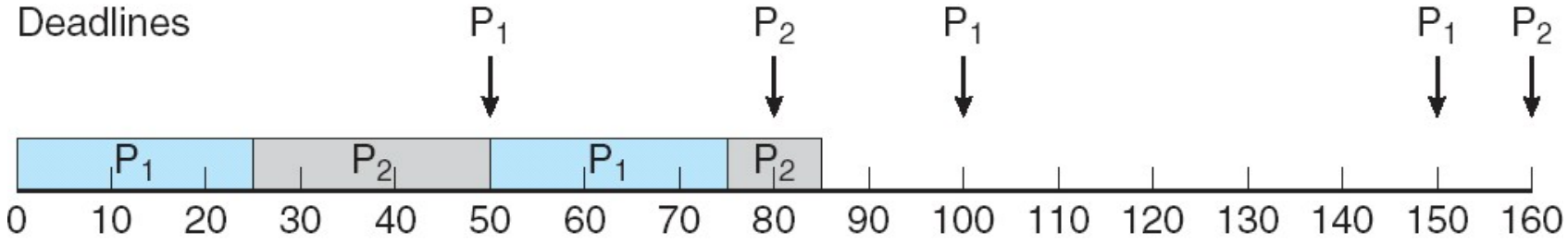
P_1 is assigned a higher priority than P_2.







Missed Deadlines with Rate Monotonic Scheduling



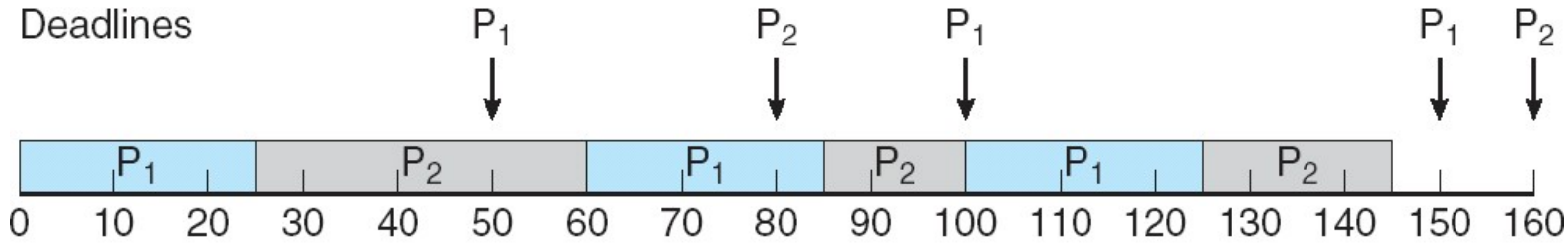


Earliest Deadline First Scheduling (EDF)

Priorities are assigned according to deadlines:

the earlier the deadline, the higher the priority;

the later the deadline, the lower the priority





Proportional Share Scheduling

T shares are allocated among all processes in the system

An application receives *N* shares where $N < T$

This ensures each application will receive ***N*** / *T* of the total processor time





POSIX Real-Time Scheduling

The POSIX.1b standard

API provides functions for managing real-time threads

Defines two scheduling classes for real-time threads:

1. SCHED_FIFO - threads are scheduled using a FCFS strategy with a FIFO queue. There is no time-slicing for threads of equal priority
2. SCHED_RR - similar to SCHED_FIFO except time-slicing occurs for threads of equal priority

Defines two functions for getting and setting scheduling policy:

1. `pthread_attr_getsched_policy(pthread_attr_t *attr, int *policy)`
2. `pthread_attr_setsched_policy(pthread_attr_t *attr, int policy)`





POSIX Real-Time Scheduling API

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[])
{
   int i, policy;
   pthread_t tid[NUM_THREADS];
   pthread_attr_t attr;
   /* get the default attributes */
   pthread_attr_init(&attr);
   /* get the current scheduling policy */
   if (pthread_attr_getsched_policy(&attr, &policy) != 0)
      fprintf(stderr, "Unable to get policy.\n");
   else {
      if (policy == SCHED_OTHER) printf("SCHED OTHER\n");
      else if (policy == SCHED_RR) printf("SCHED RR\n");
      else if (policy == SCHED_FIFO) printf("SCHED FIFO\n");
   }
```





POSIX Real-Time Scheduling API (Cont.)

```
   /* set the scheduling policy - FIFO, RR, or OTHER */
   if (pthread_attr_setsched_policy(&attr, SCHED_FIFO) != 0)
      fprintf(stderr, "Unable to set policy.\n");
   /* create the threads */
   for (i = 0; i < NUM_THREADS; i++)
      pthread_create(&tid[i], &attr, runner, NULL);
   /* now join on each thread */
   for (i = 0; i < NUM_THREADS; i++)
      pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param)
{
   /* do some work ... */
   pthread_exit(0);
}
```





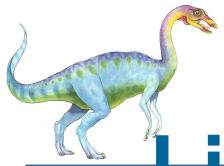
Operating System Examples

Linux scheduling

Windows scheduling

Solaris scheduling





Linux Scheduling Through Version 2.5

Prior to kernel version 2.5, ran variation of standard UNIX scheduling algorithm

Version 2.5 moved to constant order *O*(1) scheduling time

- Preemptive, priority based
- Two priority ranges: time-sharing and real-time
- **Real-time** range from 0 to 99 and **nice** value from 100 to 140
- Map into global priority with numerically lower values indicating higher priority
- Higher priority gets larger q
- Task run-able as long as time left in time slice (**active**)
- If no time left (**expired**), not run-able until all other tasks use their slices
- All run-able tasks tracked in per-CPU **runqueue** data structure
  - Two priority arrays (active, expired)
  - Tasks indexed by priority
  - When no more active, arrays are exchanged





Priorities and Time-slice length

numeric priority	relative priority		time quantum
0	highest	real-time tasks	200 ms
•			
•			
•			
99			
100		other tasks	
•			
•			
•			
140	lowest		10 ms





List of Tasks Indexed According to Priorities







Linux Scheduling in Version 2.6.23 +

Completely Fair Scheduler (CFS)

Scheduling classes

- Each has specific priority
- Scheduler picks highest priority task in highest scheduling class
- Rather than quantum based on fixed time allotments, based on proportion of CPU time
- 2 scheduling classes included, others can be added
  1. default
  2. real-time

Quantum calculated based on **nice value** from -20 to +19

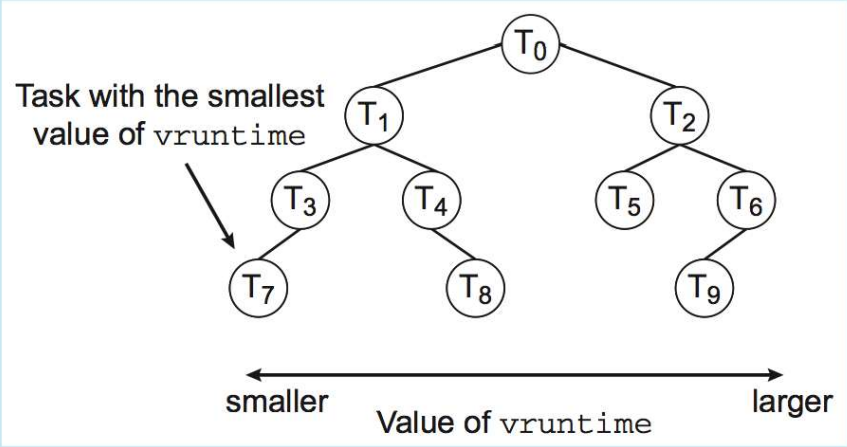
- Lower value is higher priority
- Calculates **target latency** – interval of time during which task should run at least once
- Target latency can increase if say number of active tasks increases





CFS Performance

The Linux CFS scheduler provides an efficient algorithm for selecting which task to run next. Each runnable task is placed in a red-black tree—a balanced binary search tree whose key is based on the value of `vruntime`. This tree is shown below:



When a task becomes runnable, it is added to the tree. If a task on the tree is not runnable (for example, if it is blocked while waiting for I/O), it is removed. Generally speaking, tasks that have been given less processing time (smaller values of `vruntime`) are toward the left side of the tree, and tasks that have been given more processing time are on the right side. According to the properties of a binary search tree, the leftmost node has the smallest key value, which for the sake of the CFS scheduler means that it is the task with the highest priority. Because the red-black tree is balanced, navigating it to discover the leftmost node will require $O(\lg N)$ operations (where N is the number of nodes in the tree). However, for efficiency reasons, the Linux scheduler caches this value in the variable `rb_leftmost`, and thus determining which task to run next requires only retrieving the cached value.





Linux Scheduling (Cont.)

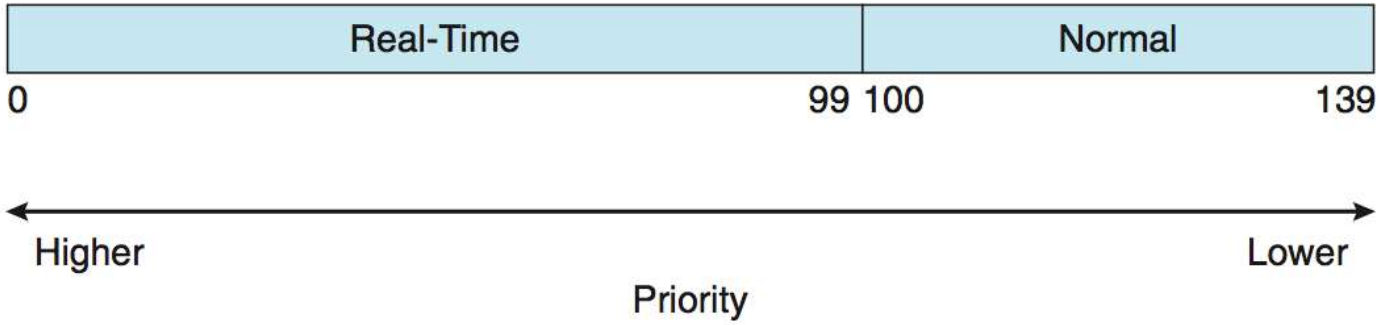
Real-time scheduling according to POSIX.1b

- Real-time tasks have static priorities

Real-time plus normal map into global priority scheme

Nice value of -20 maps to global priority 100

Nice value of +19 maps to priority 139







Windows Scheduling

Windows uses priority-based preemptive scheduling

Highest-priority thread runs next

Dispatcher is scheduler

Thread runs until (1) blocks, (2) uses time slice, (3) preempted by higher-priority thread

Real-time threads can preempt non-real-time

32-level priority scheme

Variable class is 1-15, **real-time class** is 16-31

Priority 0 is memory-management thread

Queue for each priority

If no run-able thread, runs **idle thread**





Windows Priority Classes

Win32 API identifies several priority classes to which a process can belong

- REALTIME_PRIORITY_CLASS, HIGH_PRIORITY_CLASS, ABOVE_NORMAL_PRIORITY_CLASS,NORMAL_PRIORITY_CLASS, BELOW_NORMAL_PRIORITY_CLASS, IDLE_PRIORITY_CLASS
- All are variable except REALTIME

A thread within a given priority class has a relative priority

- TIME_CRITICAL, HIGHEST, ABOVE_NORMAL, NORMAL, BELOW_NORMAL, LOWEST, IDLE

Priority class and relative priority combine to give numeric priority

Base priority is NORMAL within the class

If quantum expires, priority lowered, but never below base

If wait occurs, priority boosted depending on what was waited for

Foreground window given 3x priority boost

Windows 7 added **user-mode scheduling** (**UMS**)

- Applications create and manage threads independent of kernel
- For large number of threads, much more efficient





Windows Priorities

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1





Solaris

- Priority-based scheduling
- Six classes available
  - Time sharing (default) (TS)
  - Interactive (IA)
  - Real time (RT)
  - System (SYS)
  - Fair Share (FSS)
  - Fixed priority (FP)
- Given thread can be in one class at a time
- Each class has its own scheduling algorithm
- Time sharing is multi-level feedback queue
  - Loadable table configurable by sysadmin





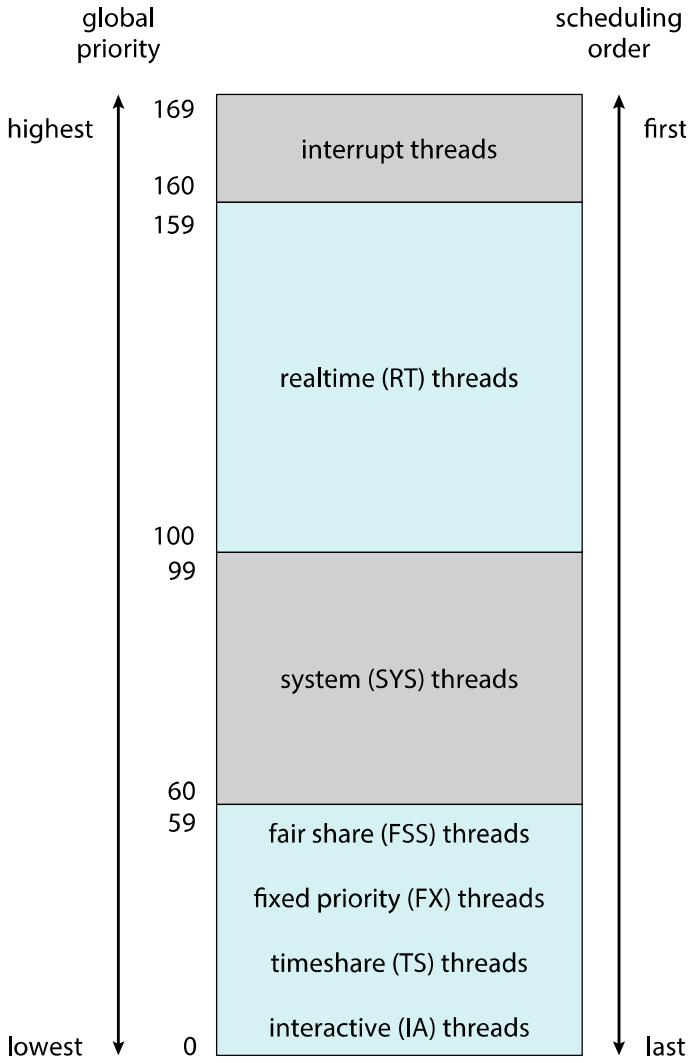
Solaris Dispatch Table

priority	time quantum	time quantum expired	return from sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59





Solaris Scheduling







Solaris Scheduling (Cont.)

Scheduler converts class-specific priorities into a per-thread global priority

- Thread with highest priority runs next
- Runs until (1) blocks, (2) uses time slice, (3) preempted by higher-priority thread
- Multiple threads at same priority selected via RR





Algorithm Evaluation

How to select CPU-scheduling algorithm for an OS?

Determine criteria, then evaluate algorithms

Deterministic modeling

Type of **analytic evaluation**

Takes a particular predetermined workload and defines the per algorithm for that workload

Consid ving at time 0:

Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12





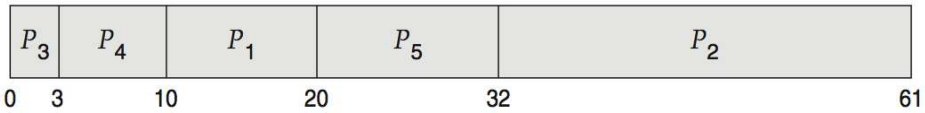
Deterministic Evaluation

For each algorithm, calculate minimum average waiting time

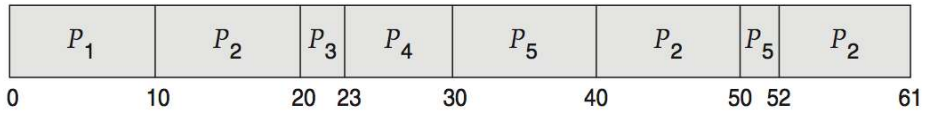
Simple and fast, but requires exact numbers for input, applies only to those i

FC





Non-preemptive SFJ is 13ms:



RR is 23ms:





Queueing Models

Describes the arrival of processes, and CPU and I/O bursts probabilistically

- Commonly exponential, and described by mean
- Computes average throughput, utilization, waiting time, etc

Computer system described as network of servers, each with queue of waiting processes

- Knowing arrival rates and service rates
- Computes utilization, average queue length, average wait time, etc





Little's Formula

n = average queue length

W = average waiting time in queue

λ = average arrival rate into queue

Little's law – in steady state, processes leaving queue must equal processes arriving, thus

$n = \lambda \times W$

Valid for any scheduling algorithm and arrival distribution

For example, if on average 7 processes arrive per second, and normally 14 processes in queue, then average wait time per process = 2 seconds





Simulations

Queueing models limited

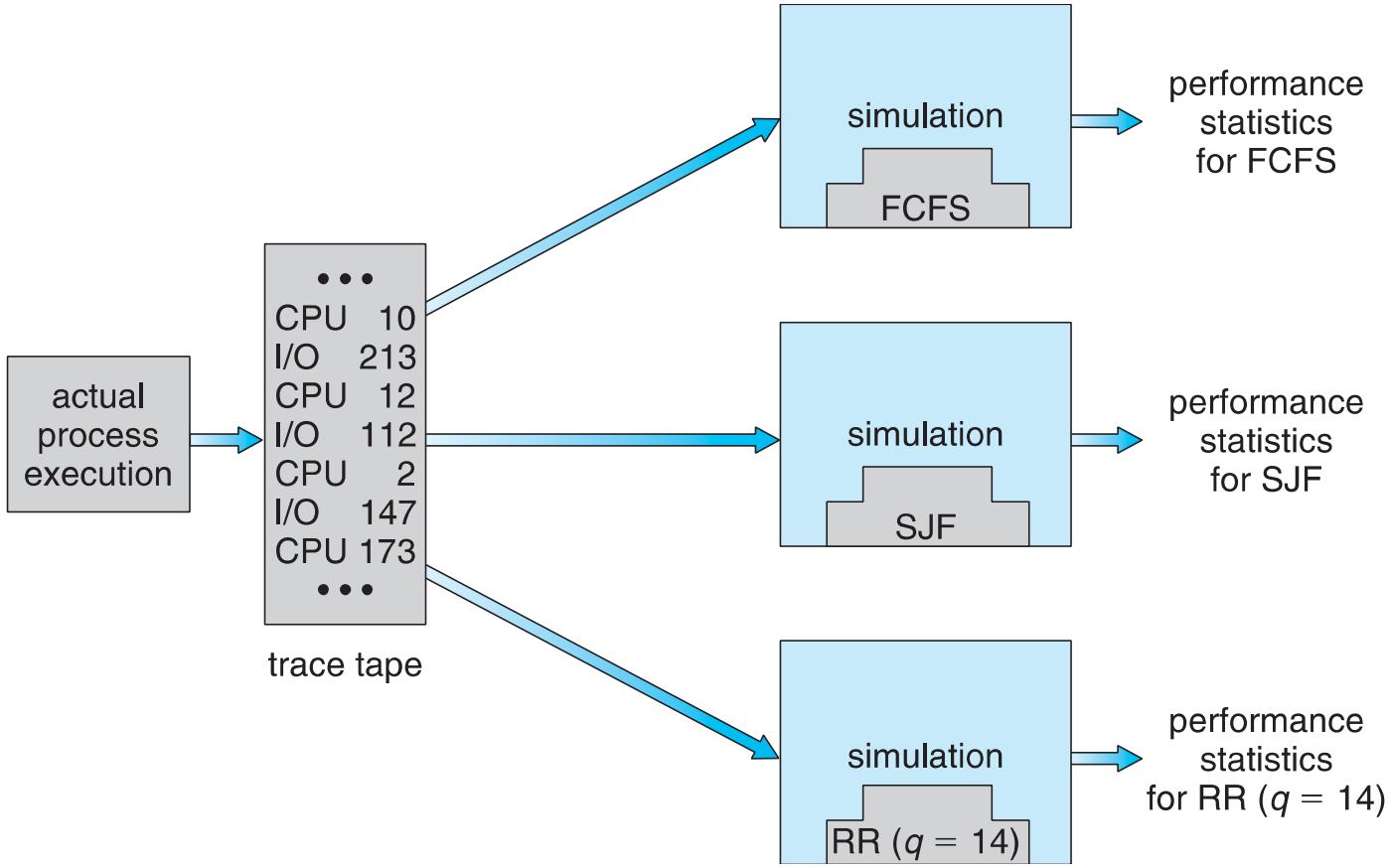
Simulations more accurate

- Programmed model of computer system
- Clock is a variable
- Gather statistics indicating algorithm performance
- Data to drive simulation gathered via
  - Random number generator according to probabilities
  - Distributions defined mathematically or empirically
  - Trace tapes record sequences of real events in real systems





Evaluation of CPU Schedulers by Simulation







Implementation

Even simulations have limited accuracy

Just implement new scheduler and test in real systems

- High cost, high risk
- Environments vary

Most flexible schedulers can be modified per-site or per-system

Or APIs to modify priorities

But again environments vary

